

GHI CHÚ: CÁC HƯỚNG NÂNG CAO CHO ĐỀ TÀI

Phần cơ bản (Diff Check + Evolution Policy + Registry + Auto-sync/Freeze) đã hoàn thiện và test với Docker thật. Đây là 5 hướng có thể đào sâu thêm nếu còn thời gian — KHÔNG cần làm hết, chỉ nên chọn 1-2 hướng phù hợp thời gian còn lại.

1. Polling → Event-driven

Hiện tại: hệ thống chỉ biết "có gì đó khác" giữa 2 lần quét (polling theo chu kỳ), không biết chính xác thời điểm đổi, và có thể bỏ lỡ thay đổi thoáng qua (thêm cột rồi xóa ngay trong cùng 1 chu kỳ).

Hướng nâng cấp: nghe trực tiếp DDL event thay vì polling — Postgres event trigger, MySQL binlog DDL, hoặc mở rộng qua CDC (Debezium/SeaTunnel). Đây là hướng nối liền được kinh nghiệm thực tế đang làm ở công ty vào đồ án — điểm cộng lớn khi bảo vệ vì cho thấy hiểu sâu hơn là chỉ code theo hướng dẫn.

2. Suy luận Rename thật (thay vì DROP + ADD)

Hiện tại: đổi tên cột bị coi là 2 thay đổi riêng biệt (DROP_COLUMN cột cũ + ADD_COLUMN cột mới) — đơn giản hoá có chủ đích từ đầu, chưa có thuật toán suy luận rename thật.

Hướng nâng cấp: heuristic đơn giản — nếu 1 cột biến mất và 1 cột mới xuất hiện cùng lúc, cùng data_type/nullable, coi là ứng viên rename, đưa lên UI cho kỹ sư xác nhận thay vì tự động kết luận.

3. Phân tích tác động hạ nguồn (Impact/Lineage Analysis)

Hiện tại: hệ thống chỉ biết "cột này đổi", không biết cột đó đang được dùng ở bao nhiêu Materialized View, dashboard Superset nào — tức là biết CÓ LỖI nhưng không biết MỨC ĐỘ ảnh hưởng thật.

Hướng nâng cấp: đây là điểm có thể khác biệt hẳn so với tool thị trường (Liquibase, Flyway...) vì có sẵn ngữ cảnh thực tế (StarRocks block table, MV, Superset) để nối vào — quét xem cột bị đổi có xuất hiện trong định nghĩa SQL của MV/view nào không, biến đề tài từ "phát hiện lỗi" thành "phát hiện + đánh giá mức độ ảnh hưởng thật".

4. Rollback cho thay đổi đã auto-sync

Hiện tại: nếu 1 thay đổi non-breaking đã được tự động áp dụng (auto-sync) xong mới phát hiện ra là sai (VD: kiểu dữ liệu mới gây vấn đề ở tầng downstream), không có cơ chế undo — chỉ có Approve/Reject cho breaking change, không có "Undo" cho non-breaking đã chạy rồi.

Hướng nâng cấp: lưu lại DDL nghịch đảo (VD: ADD_COLUMN → lưu kèm DROP COLUMN tương ứng) mỗi khi auto-sync thành công, cho phép rollback 1 version cụ thể sau này.

5. mongo.py / clickhouse.py — driver NoSQL → columnar

Hiện tại: mới chỉ có postgres.py/mysql.py (SQL-to-SQL) — bài toán đã có nhiều nghiên cứu kinh điển (kể cả PRISM đã trích dẫn ở Chương III). Use-case 2 trong Chương I (NoSQL → SQL, Mongo → ClickHouse) khó hơn hẳn về mặt học thuật vì Mongo không có schema cố định.

Hướng nâng cấp: suy luận schema từ sample N document JSON (infer type từng field), rồi đề xuất cột vật lý tương ứng bên ClickHouse — đây là phần khó hơn, ít người làm, và đúng cam kết use-case 2 đã viết trong Chương I. Nên ưu tiên nếu muốn điểm sáng tạo học thuật rõ ràng nhất trong 5 hướng này.

Kết luận

Phần cơ bản hiện tại đã đủ để chứng minh ý tưởng (proof of concept) hoạt động đúng — không bắt buộc phải làm thêm hướng nào. Nếu muốn đồ án có điểm nhấn học thuật rõ ràng để tránh bị hỏi "vậy khác gì Liquibase/Flyway", nên chọn 1 trong 2 hướng: `mongo.py/clickhouse.py` (khó hơn, đúng cam kết report) HOẶC impact analysis đơn giản (nối liền kinh nghiệm thực tế, dễ tạo ấn tượng khi bảo vệ). Không cần làm cả 5.